

Program2

Develop a Multi-Stage Dockerfile for container Orchestration

step1: project structure:

```
|--Program2
  |--src
  |--dist
    |--index.js
  |--Dockerfile
  |--build
  |--package.json
  |--package-lock.json
  |--node_modules
```

step2: create a Dockerfile

step3: create index.js inside index.js

step4: create package.json

```
i sha@meow:~/DevOps/program2$ nano Dockerfile
i sha@meow:~/DevOps/program2$ cd src
i sha@meow:~/DevOps/program2/src$ nano index.js
i sha@meow:~/DevOps/program2/src$ cd ..
i sha@meow:~/DevOps/program2$ █
```

run >> npm init -y

```
isha@meow:~/DevOps/program2$ npm init -y
Wrote to /home/isha/DevOps/program2/package.json:

{
  "name": "program2",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node src/index.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.1.0"
  },
  "devDependencies": {},
  "description": ""
}
```

step5: >>npm install express

>>npm install

```
isha@meow:~/DevOps/program2$ npm install express
removed 2 packages, changed 5 packages, and audited 67 packages in 9s

18 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
isha@meow:~/DevOps/program2$ npm install
up to date, audited 67 packages in 1s

18 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
isha@meow:~/DevOps/program2$
```

step 6 : build the dockerfile

>> docker build -t program2 .

```
isha@meow:~/DevOps/program2$ docker build -t program2 .
[+] Building 14.4s (14/14) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 326B                                0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine  8.8s
=> [auth] library/node:pull token for registry-1.docker.io        0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                      0.0s
=> [build 1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4 0.0s
=> [internal] load build context                                  0.1s
=> => transferring context: 600.34kB                               0.0s
=> CACHED [build 2/5] WORKDIR /app                                0.0s
=> [build 3/5] COPY package*.json ./                              0.0s
=> [build 4/5] RUN npm install                                    4.7s
=> [build 5/5] COPY . .                                           0.3s
=> [stage-1 3/5] COPY --from=build /app/package*.json ./         0.0s
=> [stage-1 4/5] COPY --from=build /app/src ./src                 0.0s
=> [stage-1 5/5] COPY --from=build /app/node_modules ./node_modules 0.1s
=> exporting to image                                             0.1s
=> => exporting layers                                             0.1s
=> => writing image sha256:dfc73a57a22b975a32cd622029e461d1e936679d71279 0.0s
=> => naming to docker.io/library/program2                        0.0s
```

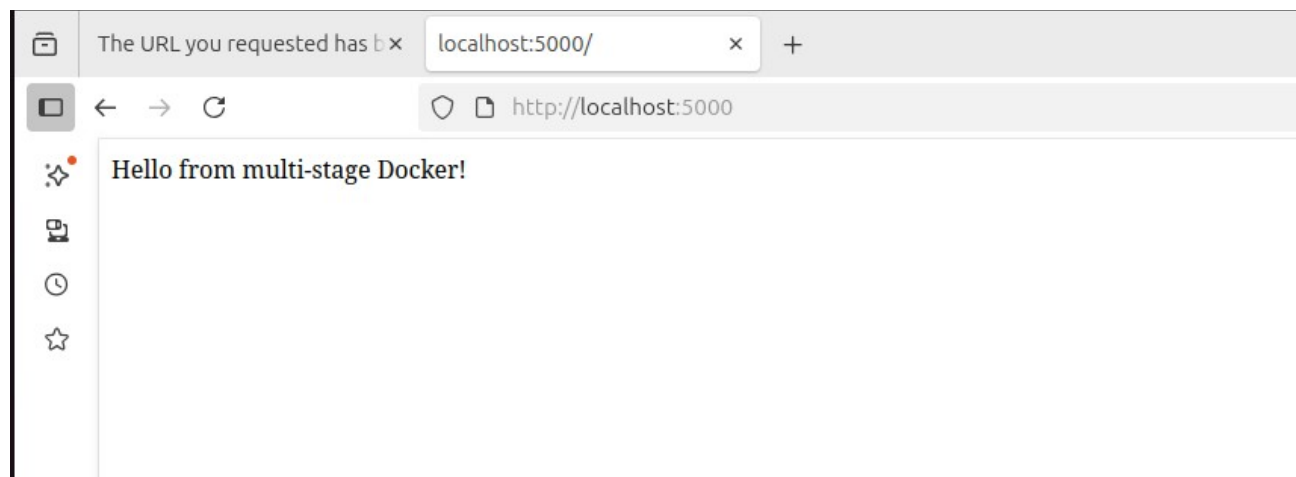
Step7: expose to the port and check the output in browser

>> docker run -p 5000:5000 program2

```
isha@meow:~/DevOps/program2$ docker run -p 5000:5000 program2
```

```
> program2@1.0.0 start
> node src/index.js
```

```
Server running on port 8080
```



>>nano Dockerfile

```
GNU nano 7.2 Dockerfile
# =====
# Stage 1: Builder Stage
# =====
FROM node:20-alpine AS builder

WORKDIR /app

# Copy package files and install dependencies
COPY package.json package-lock.json ./
RUN npm install

# Copy the rest of the app (including dist/)
COPY . .

# =====
# Stage 2: Production Stage
# =====
FROM node:20-alpine

WORKDIR /app

# Copy package files and installed node_modules from builder
COPY --from=builder /app/package.json ./
COPY --from=builder /app/package-lock.json ./
COPY --from=builder /app/node_modules ./node_modules

# Copy app source files (including dist/)
COPY --from=builder /app ./

# Expose port
EXPOSE 5000

# Run the app
CMD ["node", "dist/index.js"]
```

>>nano dist/index.js

```
GNU nano 7.2 index.js
// index.js
const express = require('express');
const app = express();

// Use PORT from environment variable, or default to 5000
const PORT = process.env.PORT || 5000;

// Define a simple route
app.get('/', (req, res) => {
  res.send('Hello from multi-stage Docker!');
});

// Listen on all network interfaces
app.listen(PORT, '0.0.0.0', () => {
  console.log(`Server running on port ${PORT}`);
});
```

